

TemSeg Overview Draft 1

Abstract—Quantitative characterization of nanoparticle size and shape distributions from Transmission Electron Microscopy (TEM) images remains a critical bottleneck in materials science workflows. Existing approaches rely on manual annotation on legacy software that trades accuracy for throughput. We present TEMseg, an open-source desktop application that combines deep learning-based instance segmentation with interactive refinement and automated analysis.

The application supports multiple domain finetuned models varied across sizes for flexibility and provides a human-in-the-loop workflow where researchers can refine segmentation results through direct contour manipulation, single-click particle splitting, and region-based re-segmentation. Per-particle morphological statistics are computed from exact pixel masks with automatic physical unit calibration from microscope metadata. TEMseg ships as a self-contained native application requiring no programming expertise, reducing analysis time from hours of manual work to minutes of interactive review.

I. FEATURES

A. Image Upload

The application supports two common image upload mechanisms: users may either click the image workspace to open the system file picker or drag and drop files directly into the upload area. Currently supported formats include TIF/TIFF, EMD, JPG/JPEG, PNG, and NumPy array files. Where available, metadata embedded within the image file such as pixel size and scale information is automatically extracted and stored for downstream analysis. Clicking the application logo clears the current workspace, removes the loaded image, and resets the workflow state.

B. Choosing models

The initial goal of this tool was to provide accessible support for state-of-the-art particle segmentation models, especially for users without dedicated GPU hardware. At present, the application provides access to two segmentation pipelines as can be seen in Fig. 2 :

a) *YoloSAM by Ardra Genc et al.*:

YoloSAM is a dual-model segmentation pipeline that combines YOLOv8 for object detection with Segment Anything Model (SAM) for dense segmentation. YOLOv8 first detects regions of interest by generating bounding boxes, which are then passed to SAM to generate instance-segmented particle masks. The models were fine-tuned by Genc et al. on a custom annotated TEM dataset, described in detail in their work. We use their published YOLOv8 weights together with the upstream SAM ViT-B checkpoint without modification. Our contribution focuses on improving runtime efficiency of the original pipeline while preserving output quality and segmentation accuracy. Three main optimizations were introduced:

1) *ONNX Runtime acceleration for YOLO inference*

The original PyTorch YOLOv8 model was exported once to ONNX format during packaging. Inference is then executed through ONNX Runtime, which performs graph-level optimizations such as operator fusion and constant folding while using CPU-optimized kernels. On our 31-image CPU benchmark, this reduced YOLO inference time from 1.52 ± 0.15 s to 1.09 ± 0.15 s (approximately $1.4 \times$ faster) without affecting detection outputs.

2) *Persistent SAM predictor reuse*

In the reference implementation, the SAM checkpoint and predictor are reloaded for every segmentation call. In our implementation, the predictor is initialized once at process startup and reused across images and segmentation runs. This removes repeated checkpoint loading and device transfer overhead, reducing end-to-end runtime from 7.11 ± 3.84 s to 6.05 ± 2.59 s per image.

3) *Caching of SAM image embeddings*

The SAM image encoder, based on a ViT-B backbone, dominates runtime cost. During interactive workflows, users frequently re-run segmentation on the same image while adjusting parameters. Instead of recomputing image embeddings each time, our pipeline caches the encoded image features and reuses them for subsequent runs. Only the lightweight decoder stage is recomputed. This reduces repeat-call SAM runtime from 4.48 ± 2.28 s to 1.43 ± 1.70 s ($3.1 \times$ faster), and overall re-run time from 5.98 ± 2.32 s to 2.47 ± 1.70 s ($2.4 \times$ faster).

All implementation related improvement are summarized in the Table I

Despite these optimizations, segmentation quality remains effectively unchanged. The median mask IoU between our implementation and the reference pipeline is 0.98, while IoU against manually annotated ground truth is 0.841 ± 0.062 for our implementation compared to 0.840 ± 0.064 for the reference implementation. All benchmarks were performed on CPU-only Apple Silicon hardware across four ground-truth images and twenty-seven additional TEM images. Raw benchmark CSV files, aggregation scripts, and evaluation harnesses are included in the supplementary materials.

b) *MaskRCNN*:

We also trained a custom Mask R-CNN model as part of this work. The primary goal was not to achieve state-of-the-art segmentation accuracy, but rather to demonstrate the feasibility of a modular “plug-and-play” framework in which different segmentation models can be swapped dynamically within the application.

The model was trained entirely on a synthetic dataset generated procedurally for this project. Synthetic TEM-like images were created by approximating backgrounds with Gaussian noise and generating particle-like structures from simple geometric primitives such as circles, ellipses, and cylinders. Shape perturbations and standard augmentation

TABLE I
ENGINEERING DELTAS BETWEEN THE ORIGINAL YOLOSAM PIPELINE AND THE TEMSEG IMPLEMENTATION. NUMBERS FROM 31 TEM IMAGES ON CPU (APPLE SILICON).

Category	Original (Genc et al.)	TemSeg implementation
YOLOv8 inference backend	Detector weights served through a general-purpose deep-learning framework in dynamic (eager) execution mode. Measured: 1.52 ± 0.15 s	Detector weights exported to a static inference graph at packaging time and served through a dedicated inference runtime, which applies ahead-of-time graph optimisation and hardware-tuned kernels. Measured: 1.09 ± 0.15 s per image, $1.4 \times$ faster.
SAM predictor lifecycle	The segmentation model is reloaded from disk and transferred onto the compute device on every segmentation call. Measured: 7.11 ± 0.18s	The segmentation model is loaded once at process start and reused across all subsequent calls; weight loading and device-transfer cost is paid exactly once. Measured: 6.05 ± 0.23 s (removes ≈ 1s end-to-end).
SAM image-embedding cache	No cache. The image encoder is by far the dominant cost of the segmentation stage and runs from scratch on every call, including when the user re-segments the same image during interactive refinement. Measured: segmentation stage 4.48s and end-to-end rerun 5.98s	Encoded image features are returned alongside the segmentation result and threaded back into the next call on the same image, bypassing the encoder entirely; only the prompt-conditioned mask decoder runs on reruns. Measured: segmentation stage 1.43s on rerun ($3.1 \times$); end-to-end 2.47s ($2.4 \times$).

techniques including contrast variation, scaling, rotation, and deformation were then applied to improve generalization.

Although the resulting model performs substantially worse than YoloSAM in terms of segmentation quality, it offers significantly lower inference latency and provides a useful lightweight alternative for rapid experimentation and interactive workflows.

On the same four hand-annotated ground-truth images, the model achieved a mean runtime of 1.62 ± 0.73 s per image on CPU, approximately $4 \times$ faster than the first-call runtime of YoloSAM. However, segmentation accuracy was both lower and less stable, with a mask IoU against ground truth of 0.696 ± 0.195 , compared to 0.841 ± 0.062 for YoloSAM.

Performance degradation was most pronounced on densely populated TEM images containing large numbers of small, visually similar particles. On the most challenging benchmark image (≈ 300 densely packed particles), Mask R-CNN achieved an IoU of 0.39, while YoloSAM maintained an IoU of 0.85. This is consistent with the known limitations of Mask R-CNN on small, crowded, and near-identical object instances, which are characteristic of nanoparticle TEM imagery.

Based on these results, YoloSAM is used as the primary segmentation pipeline within the application, while Mask R-CNN serves as a lightweight secondary option prioritizing inference speed over segmentation fidelity

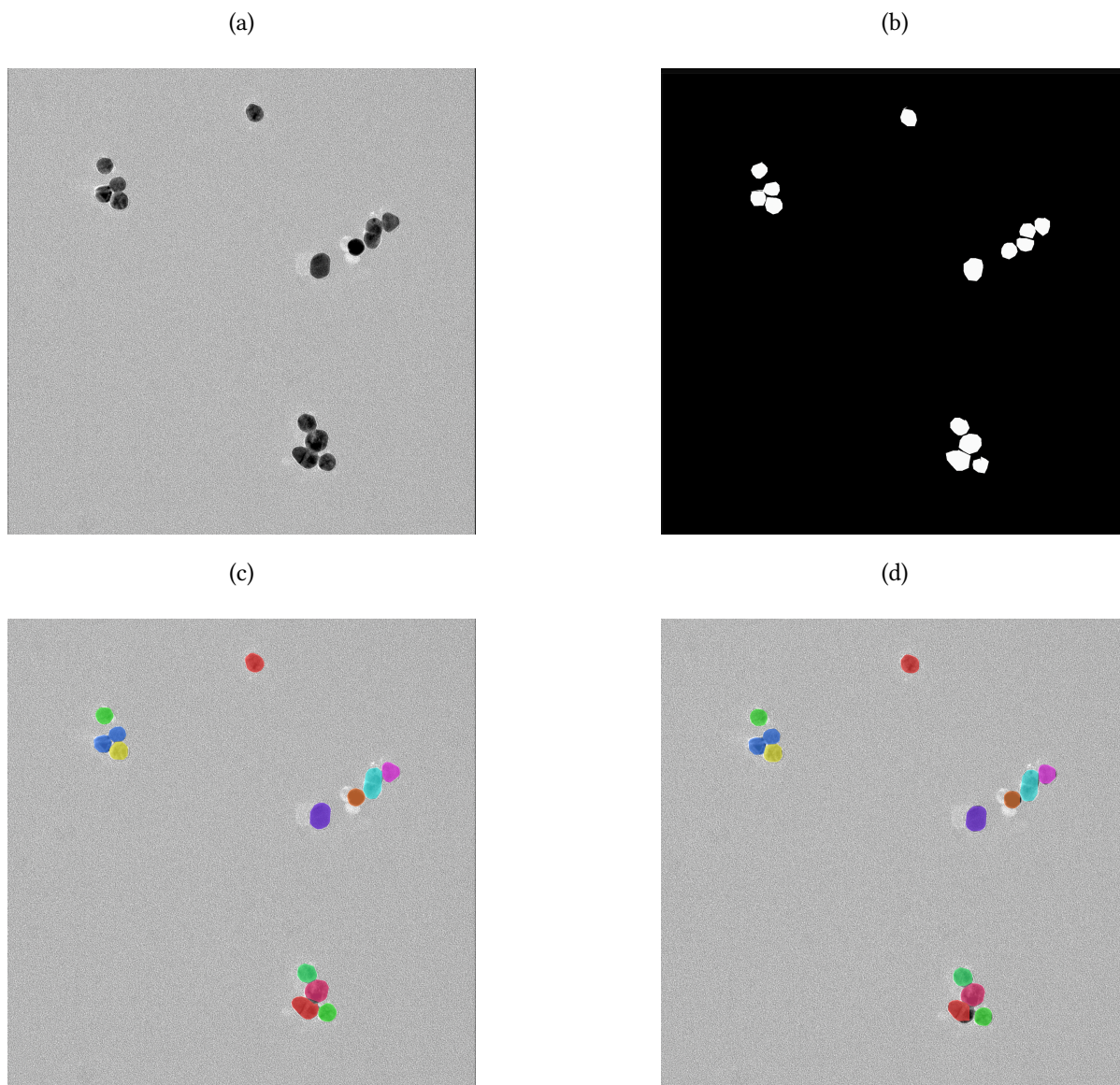


Fig. 1.
(a) Original TEM micrograph.
(b) Manually annotated ground truths.
(c) Segmentation mask using YoloSAM overlaid with colored instance labels.
(d) Segmentation mask using MaskRCNN overlaid with colored instance labels.

C. Segmentation

Once an image has been loaded and a segmentation model has been selected, the user may start inference by clicking the Run Segmentation button located in the right sidebar. After processing completes, the generated segmentation masks are automatically overlaid on top of the original image.

Users may toggle the visibility of these overlays at any time using the Show / Hide Masks control. This allows rapid comparison between the raw image and the model predictions without altering the underlying segmentation results.

Standard image navigation operations are fully supported during analysis, including zooming and panning across the image canvas. The current zoom level is displayed near the

top-right status area, and clicking the zoom percentage resets the viewport back to the default centered view.

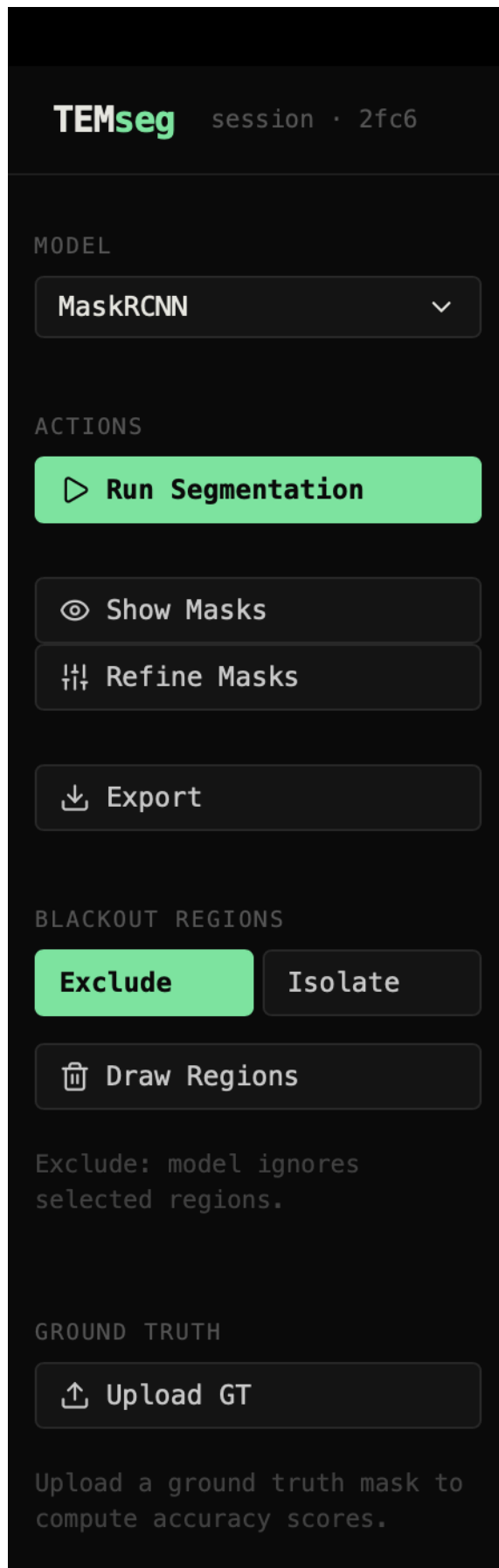


Fig. 2. Left sidebar

D. Refinement Mode

Although the provided segmentation models achieve strong performance, their outputs are still constrained by the limitations of automated inference and are not always perfect. To address this, the application provides an interactive refinement mode that allows users to correct segmentation masks without restarting the workflow.

Clicking the Refine Mode button switches the application into an editing interface where all generated masks are converted into editable polygons. Individual polygon vertices can be moved directly to better align masks with particle boundaries. Additional vertices may be inserted by clicking along a polygon edge, allowing users to represent more complex geometries, while double-clicking a vertex removes it.

Each polygon also contains a movable centroid handle that enables rotation and orientation adjustment. Standard editing shortcuts are supported: pressing Delete or Backspace removes the selected polygon, while Ctrl+Z and Ctrl+Y provide undo and redo functionality. These tools enable efficient correction of common segmentation errors, such as slight over-segmentation or under-segmentation around particle boundaries.

For larger errors such as severely incorrect masks or completely missed particles, users may duplicate an existing polygon with a similar shape using copy-paste operations (Ctrl+C / Ctrl+V). The duplicated polygon can then be repositioned and refined manually to fit the target particle. This significantly reduces the effort required to annotate missing structures from scratch.

A particularly common failure case in TEM nanoparticle analysis occurs when adjacent particles appear fused together with weak or indistinguishable boundaries. In these cases, the model may incorrectly segment multiple particles as a single instance. To address this, the application includes a Split Instance mode accessible from the sidebar. In this mode, users place a set of foreground marker points over the desired particle regions. These points are then passed to Segment Anything Model (SAM) as prompt inputs, allowing the selected particle region to be re-segmented into multiple instances. The resulting masks may then be further refined manually if required.

Because refinement operations are performed interactively, changes must be explicitly saved using the Save Refinements option in the sidebar. Exiting refinement mode without saving discards all unsaved edits.

Overall, refinement mode provides a practical balance between automated segmentation and human correction, enabling users to efficiently achieve the level of accuracy required for downstream quantitative analysis.